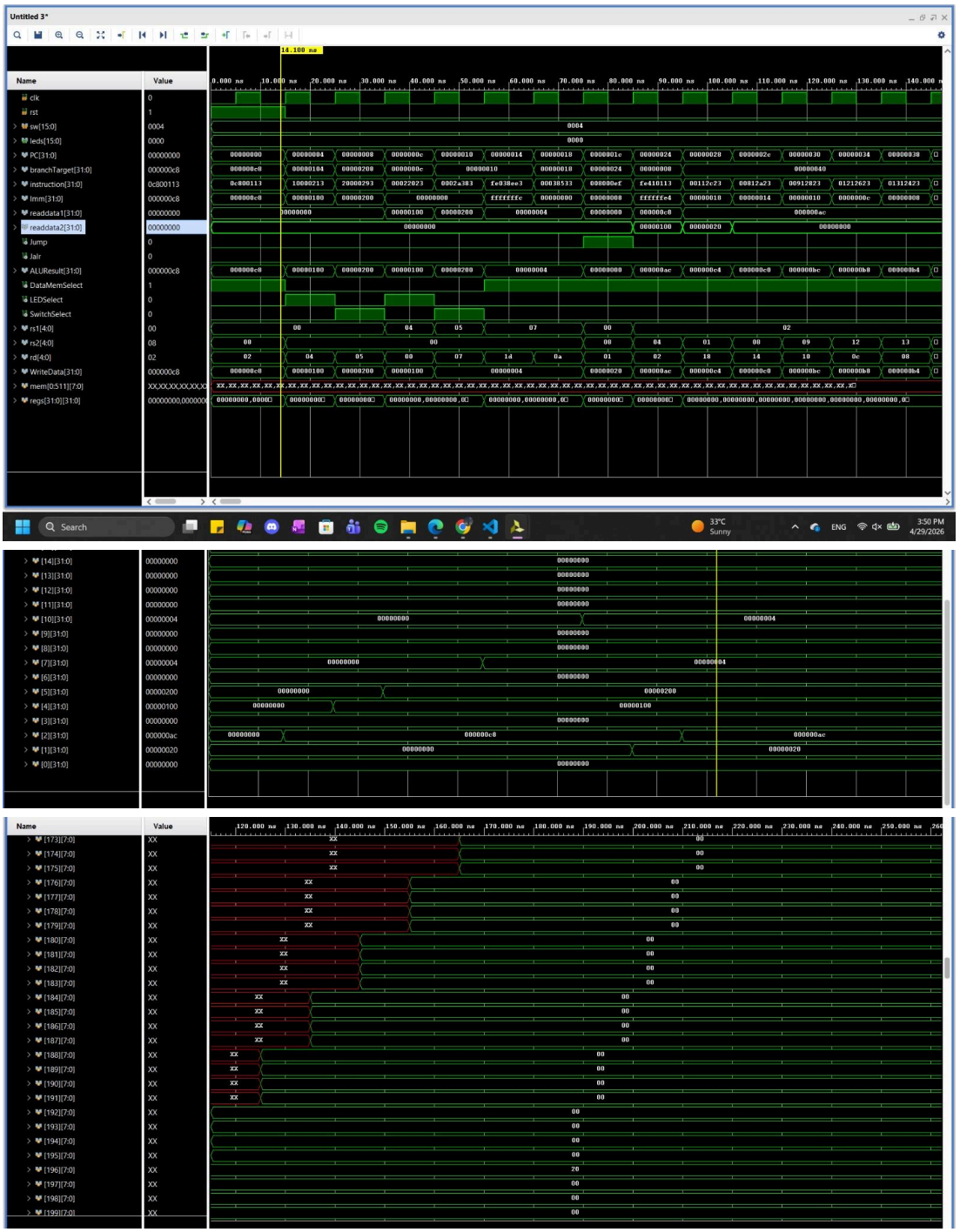
Task C:

This RISC-V assembly program implements a factorial calculator as an FSM running on a single-cycle processor, using memory-mapped I/O to display intermediate and final results on the Basys 3 FPGA board. The program initializes the stack pointer, LED address at 256, and switch address at 512, then enters an idle state where it continuously polls the switches until a non-zero value is detected. Once a non-zero switch value is read, it is passed as the input argument *n* into the factorial subroutine via register x10.

Inside the subroutine, the program iterates from 1 up to n, computing the factorial step by step using a nested multiply loop that performs multiplication through repeated addition. At each step of the outer loop, the current iterator value i and the running factorial result are packed together into a single 16-bit value by shifting i left by 12 bits and OR-ing it with the lower 12 bits of the result, which is then written to the LED address so the FPGA simultaneously displays both the current step and the running product. The stack is used extensively to save and restore seven registers including the return address, ensuring the subroutine returns cleanly to the idle state after completion.

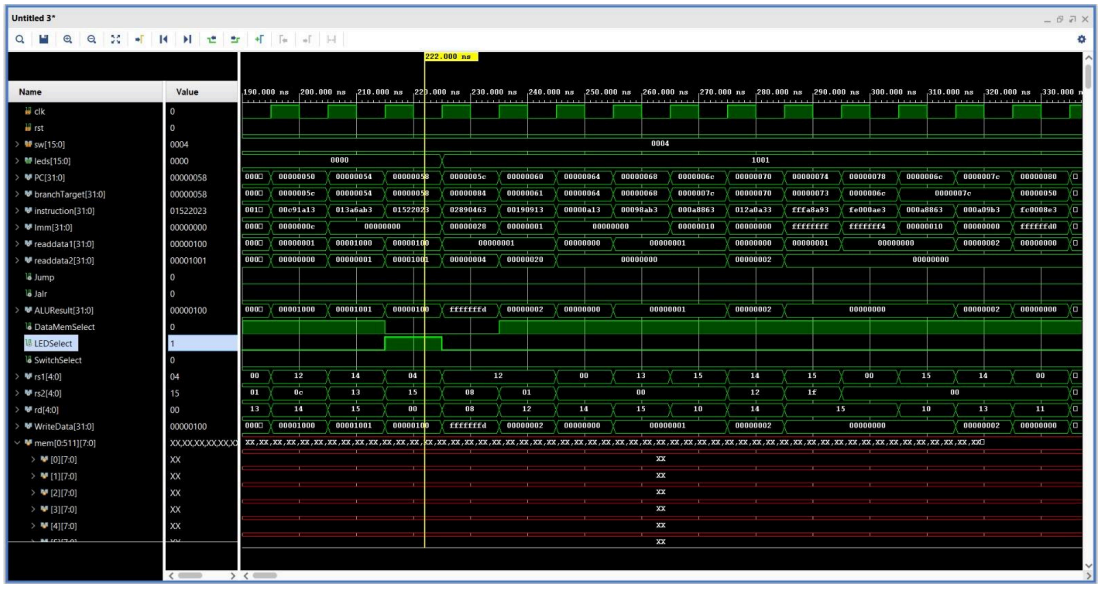
Test Bench:



❖ Task C begins by initializing the main registers. Register x2 is set to 200 and is used as the stack pointer. Register x4 stores 256, the memory-mapped LED/display base address, and register x5 stores 512, the switch input base

address. The instruction sw x0, 0(x4) clears the LEDs by writing 0 to address 256.

- The processor then reads the switch value using lw x7, 0(x5). In this testbench, the switches are set to 4, so register x7 stores the value 4, which is also visible in the register file screenshot. The instruction add x10, x7, x0 then copies this switch value into x10, which is used as the input argument n for the factorial function.
- Next, the jal x1, factorial instruction calls the factorial subroutine. This saves the return address, PC + 4, into register x1 and jumps to the factorial function at instruction address 36. In this case, x1 stores 32, which is the address of the instruction after the jal.



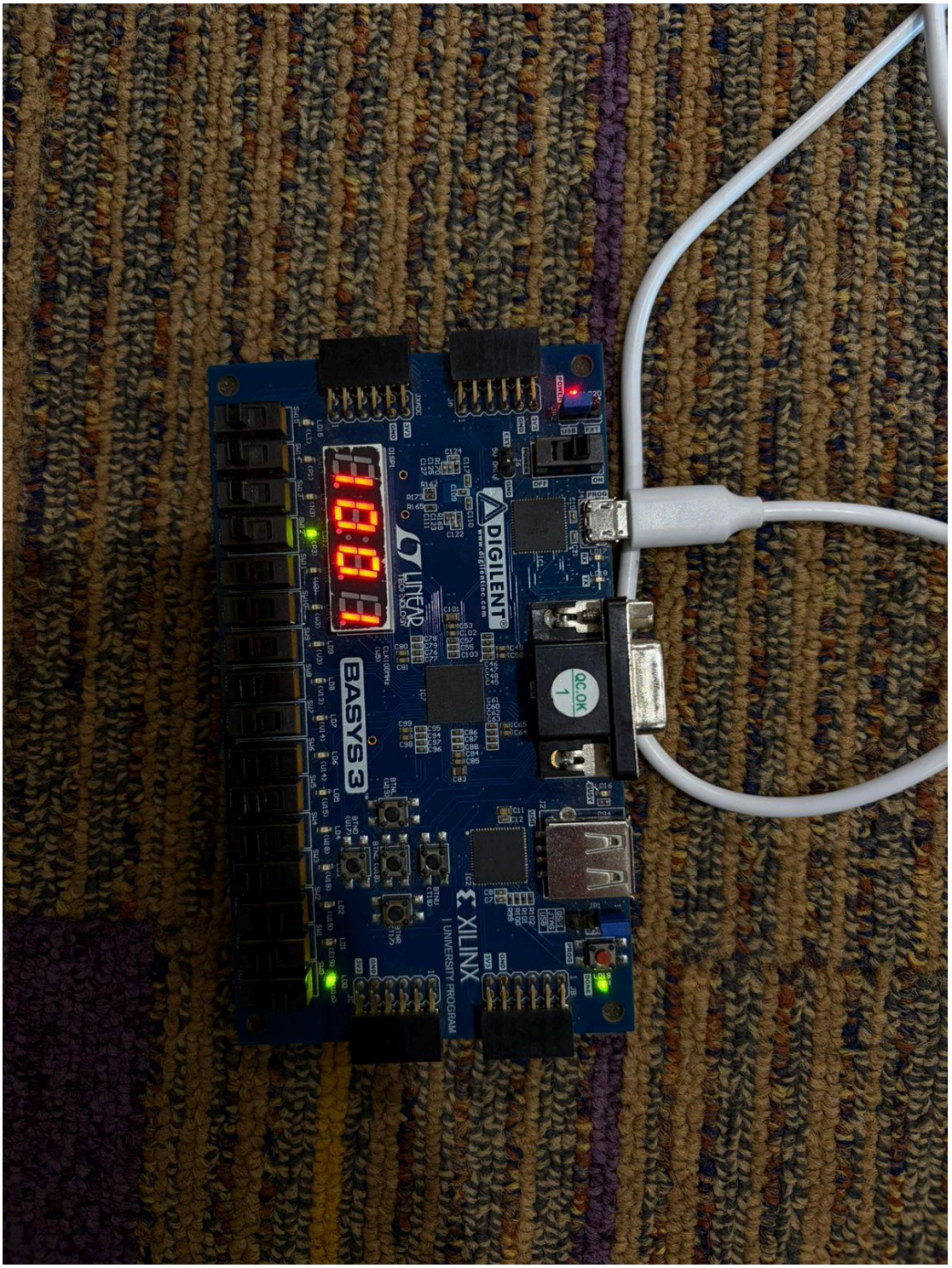
- Inside the factorial function, stack space is created using addi x2, x2, -28, so the stack pointer moves from 200 to 172. This creates space to save seven registers. The return address in x1 is stored at memory address 196, since 172 + 24 = 196. The temporary and saved registers are also stored on the stack: x8 at 192, x9 at 188, x18 at 184, x19 at 180, x20 at 176, and x21 at 172. At this point, these saved register values are initially 0.

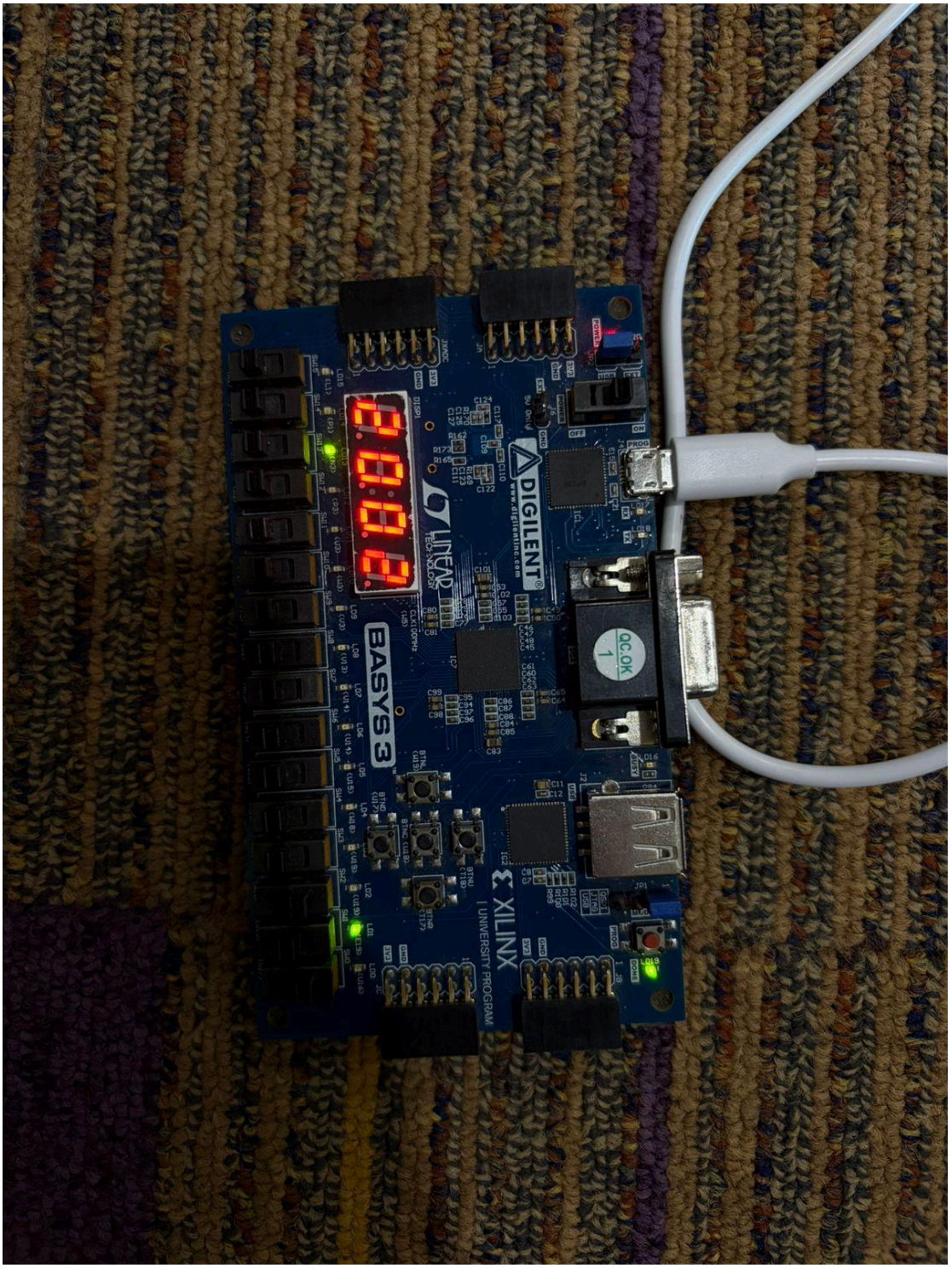
- The factorial calculation then begins. Register x8 stores the input value n = 4, register x18 is initialized to 1 as the loop counter i, and register x19 (holds the running factorial value) is initialized to 1 as the factorial result. During each loop iteration, the program packs the display value as [i << 12] | result. This means the upper 4 bits show the current value of i, while the lower 12 bits show the current factorial result.

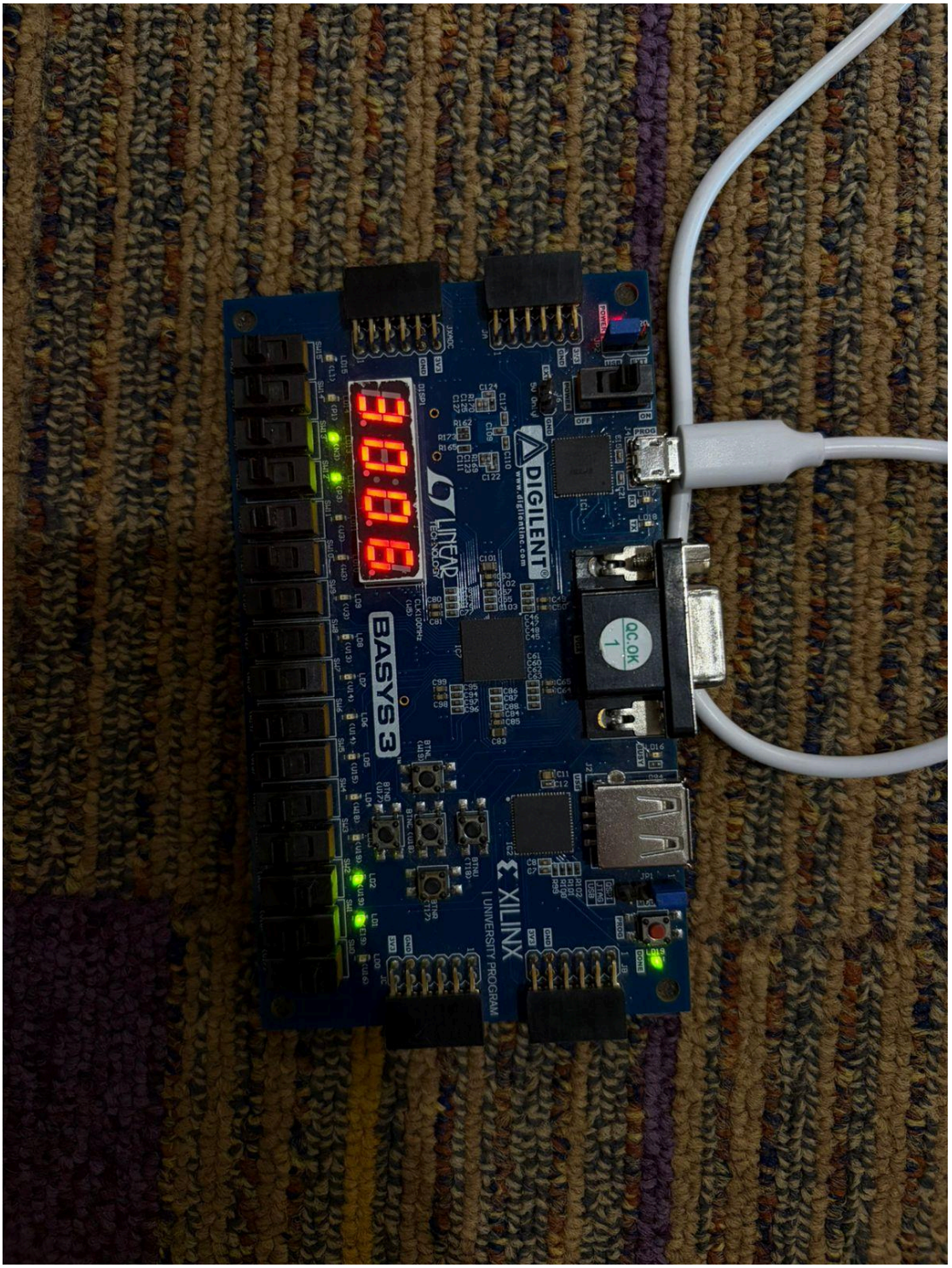
- At PC = 0x7C, which corresponds to instruction address 124, the updated factorial result is written into register x19. Register x19 is the register used to store the running factorial result throughout the program.
- At this point, the ALU result contains the next factorial value, and the write data signal shows the same value being written back. The destination register rd is 13 in hexadecimal, which corresponds to register x19.
- When the PC reaches 0x58, the program executes the store instruction that sends the packed display value to the LED/display address. At this stage, the LED select signal goes high, enabling the output to be shown on the LEDs. This confirms that the calculated factorial value is being correctly stored in x19, packed with the loop counter, and displayed through the LED output.

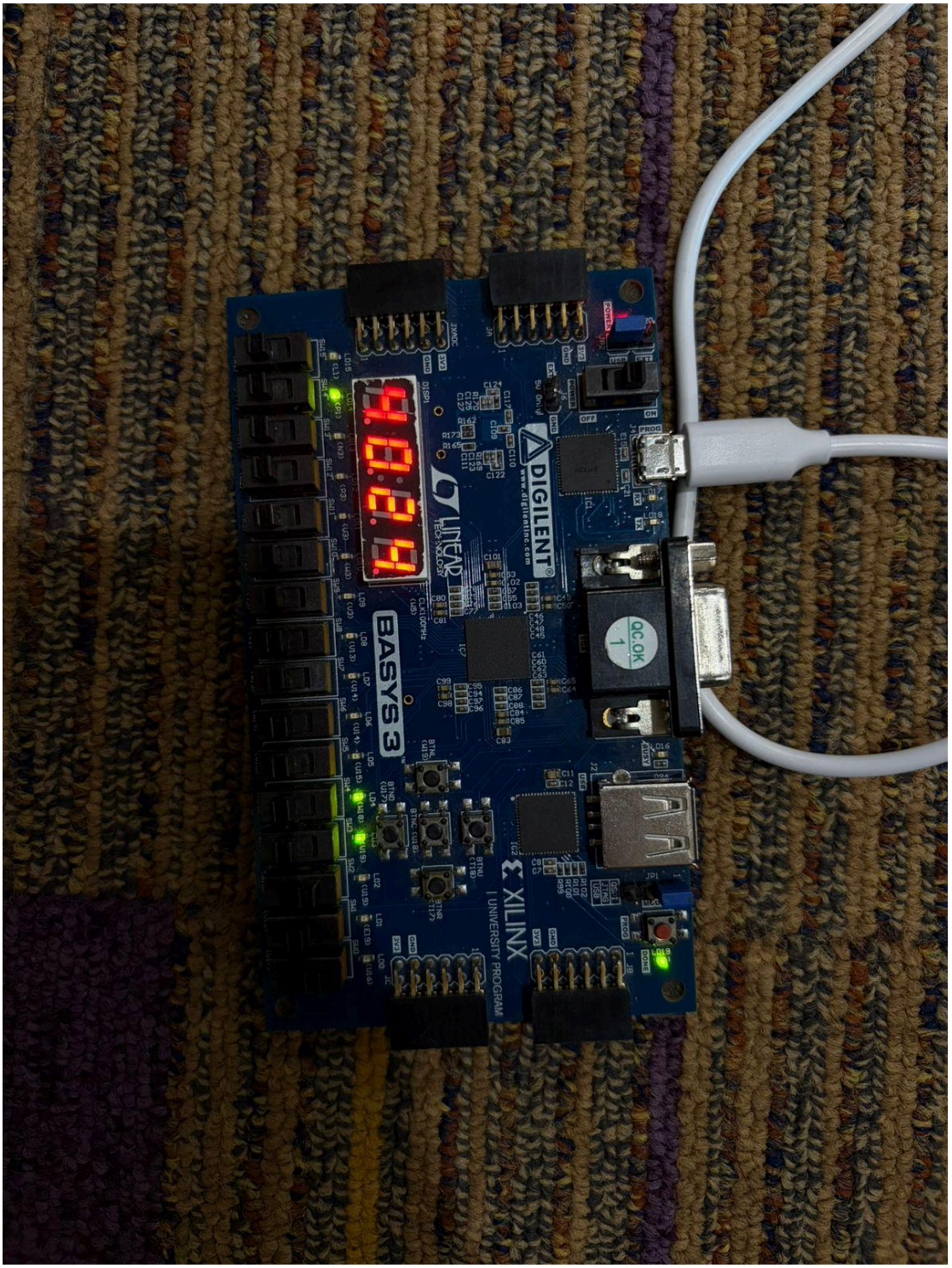
Results of Task C:

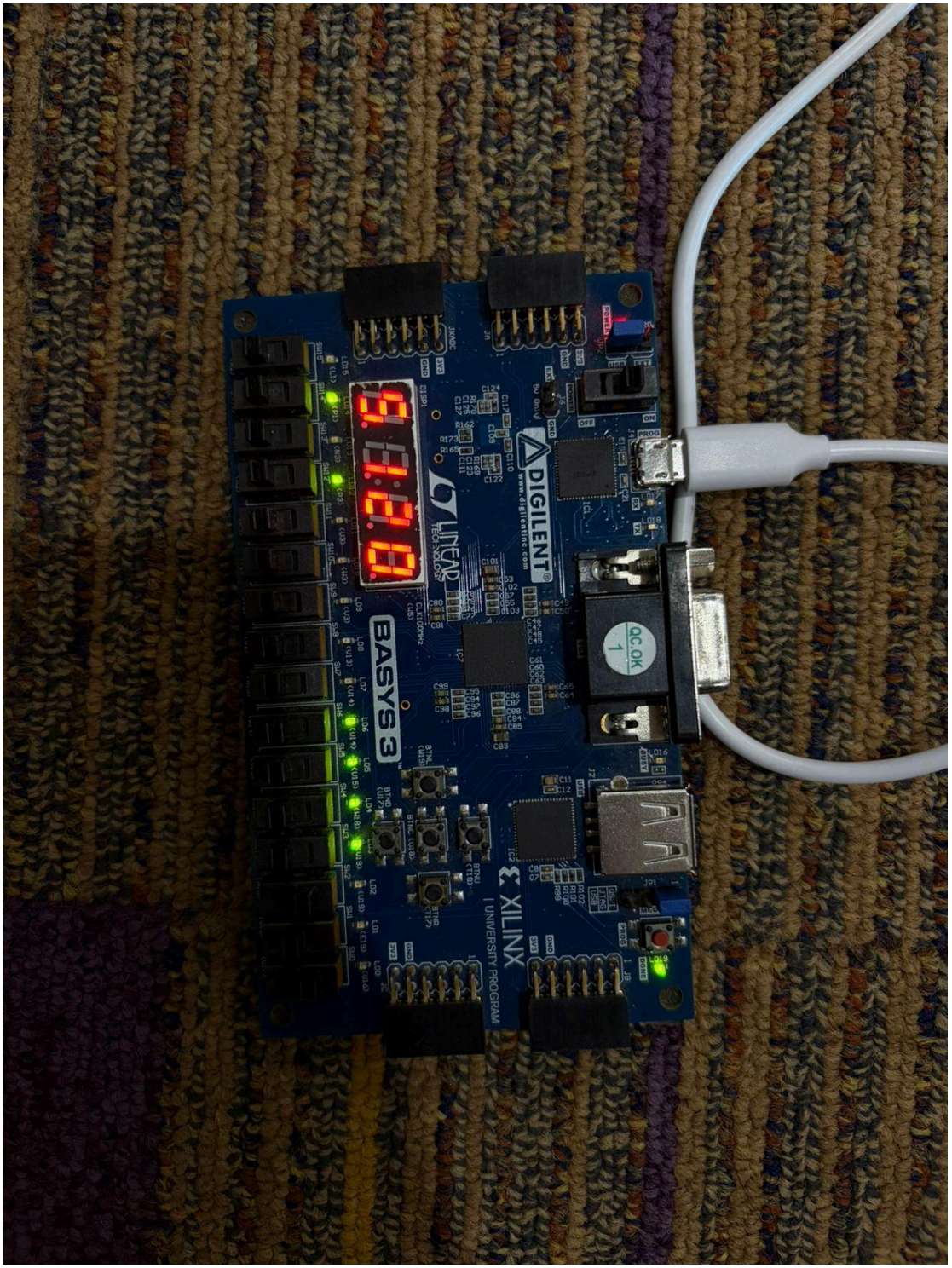
For an input of 5, the final packed LED value written is 0x5078, where the upper 4 bits represent i equal to 5 and the lower 12 bits represent 120, which is 5 factorial.


DIGILENT
www.digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM
QC.OK


BASYS 3
DIGILENT
www.digilentinc.com
LINEAR TECHNOLOGY
XILINX
UNIVERSITY PROGRAM
qc.ok


DIGILENT
www.digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM
QC.OK


DIGILENT
www.digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM
qc.ok


DIGILENT
www.digilentinc.com
LINEAR TECHNOLOGY
BASYS 3
XILINX
UNIVERSITY PROGRAM
QC.OK